Recursion error when pickling unpickling Aer functions

https://github.com/Qiskit/qiskit-aer/issues/1537

ROW 52

Recursion error when pickling/unpickling Aer functions #1537

New issue

Closed as not planned

kessler-frost opened on Jun 13, 2022

What is the expected behavior?

Currently, if we try to pickle a qiskit function with `cloudpickle` we get the following error. The expected behavior is that it doesn't give this error:

```
from qiskit import Aer
from qiskit.algorithms import QAOA
import cloudpickle as pickle


def get_qaoa(optimizer):
    return QAOA(optimizer, quantum_instance=Aer.get_backend('statevector_simulator'))


pickled_get_qaoa = pickle.dumps(get_qaoa)
pickle.loads(pickled_get_qaoa)
```

Error:

```
---------------------------------------------------------------------------
RecursionError                            Traceback (most recent call last)
/Users/sankalpsanand/dev/covalent/covalent/scratch.ipynb Cell 1' in <cell line: 11>()
          return QAOA(optimizer, quantum_instance=Aer.get_backend('statevector_simulator'))
      pickled_get_qaoa = pickle.dumps(get_qaoa)
---->  pickle.loads(pickled_get_qaoa)

File ~/opt/anaconda3/envs/cova-qa-1/lib/python3.8/site-packages/qiskit/__init__.py:89, in AerWrapper.__getatt
      def __getattr__(self, attr):
---->     if not self.aer:
              try:
                  from qiskit.providers import aer

File ~/opt/anaconda3/envs/cova-qa-1/lib/python3.8/site-packages/qiskit/__init__.py:89, in AerWrapper.__getatt
      def __getattr__(self, attr):
---->     if not self.aer:
              try:
                  from qiskit.providers import aer

    [... skipping similar frames: AerWrapper.__getattr__ at line 89 (2969 times)]

File ~/opt/anaconda3/envs/cova-qa-1/lib/python3.8/site-packages/qiskit/__init__.py:89, in AerWrapper.__getatt
      def __getattr__(self, attr):
---->     if not self.aer:
              try:
                  from qiskit.providers import aer

RecursionError: maximum recursion depth exceeded
```

I think making these functions `cloudpickle`-able is a nice to have feature for remote execution.

kessler-frost added enhancement on Jun 13, 2022

mtreinish on Jun 13, 2022 Member

The issue you're hitting isn't actually an issue in Aer, it's in your reproduce code. The infinite recursion is because you're trying to serialize the lazy loader class directly by embedding `qiskit.Aer` inside a function. That class (which is part of `qiskit-terra` not `qiskit-aer`) isn't serializable by design. It's intended to be nothing but a compatibility shim that lazy loads aer to ensure an earlier API is respected without force aer to imported by terra on loading. To fix this you would need to change `Aer.get_backend('statevector_simulator'))` to `qiskit.providers.aer.backends.StatevectorSimulator()`. In other words running:

Assignees
No one assigned

Labels
enhancement

Type
No type

Projects
No projects

Milestone
No milestone

Relationships
None yet

Development
No branches or pull requests

Notifications Customize

Subscribe

You're not receiving notifications from this thread.

Participants

kessler-frost added enhancement on Jun 13, 2022

mtreinish on Jun 13, 2022 Member

The issue you're hitting isn't actually an issue in Aer, it's in your reproduce code. The infinite recursion is because you're trying to serialize the lazy loader class directly by embedding `qiskit.Aer` inside a function. That class (which is part of `qiskit-terra` not `qiskit-aer`) isn't serializable by design. It's intended to be nothing but a compatibility shim that lazy loads aer to ensure an earlier API is respected without force aer to imported by terra on loading. To fix this you would need to change `Aer.get_backend('statevector_simulator'))` to `qiskit.providers.aer.backends.StatevectorSimulator()`. In other words running:

```
from qiskit.providers.aer import backends
from qiskit.algorithms import QAOA
import pickle


def get_qaoa(optimizer):
    return QAOA(optimizer, quantum_instance=backends.StatevectorSimulator())


pickled_get_qaoa = pickle.dumps(get_qaoa)
pickle.loads(pickled_get_qaoa)
```

works locally for me.

That being said do note that backend objects are generally not serializeable as they often contain things like async execution handles or other objects which can't be serialized. Aer works because we have wrappers around the compiled extensions that can handle this for us.

I'm going to close this issue as I don't think there is anything else to do here. But please feel free to reopen it if I'm missing something or there is more to discuss on this.

1

mtreinish closed this as not planned on Jun 13, 2022

kessler-frost mentioned this on Jun 14, 2022

Cloudpickle fails to preserve torch gradients AgnostiqHQ/covalent#649

Cloudpickle fails to preserve torch gradients #649

New issue

Closed

cjao opened on Jun 14, 2022

We use cloudpickle everywhere to serialize and deserialize data. Unfortunately Cloudpickle seems to lose crucial properties of Torch Tensors, such as gradients:

```
import torch
from dask.distributed.protocol import serialize, deserialize

def loss(x):
    l = torch.sum(x**2)
    return l

x = torch.tensor([2.0], requires_grad=True)
cost = loss(x)

cost.backward()

print(x, type(x), x.grad)
```

```
tensor([2.], requires_grad=True) <class 'torch.Tensor'> tensor([4.])
```

Now let's try pickling and unpickling:

```
import cloudpickle as pickle

y = pickle.loads(pickle.dumps(x))
print(x, type(x), x.grad)
print(y, type(y), y.grad)
```

```
tensor([2.], requires_grad=True) <class 'torch.Tensor'> tensor([4.])
tensor([2.], requires_grad=True) <class 'torch.Tensor'> None
```

cjao added priority / high on Jun 14, 2022

cjao on Jun 14, 2022 — Contributor — Author

Dask has special handling for Torch Tensors and other popular data structures:
https://github.com/dask/distributed/tree/177dfb891089cc872bab34fb8369d75cae13bc0a/distributed/protocol

We may need to extend cloudpickle similarly. See also this issue

kessler-frost on Jun 14, 2022 — Member

Similar issue:
Qiskit/qiskit-aer#1537

Assignees
No one assigned

Labels
Team East | Team West | priority / high

Type
No type

Projects
No projects

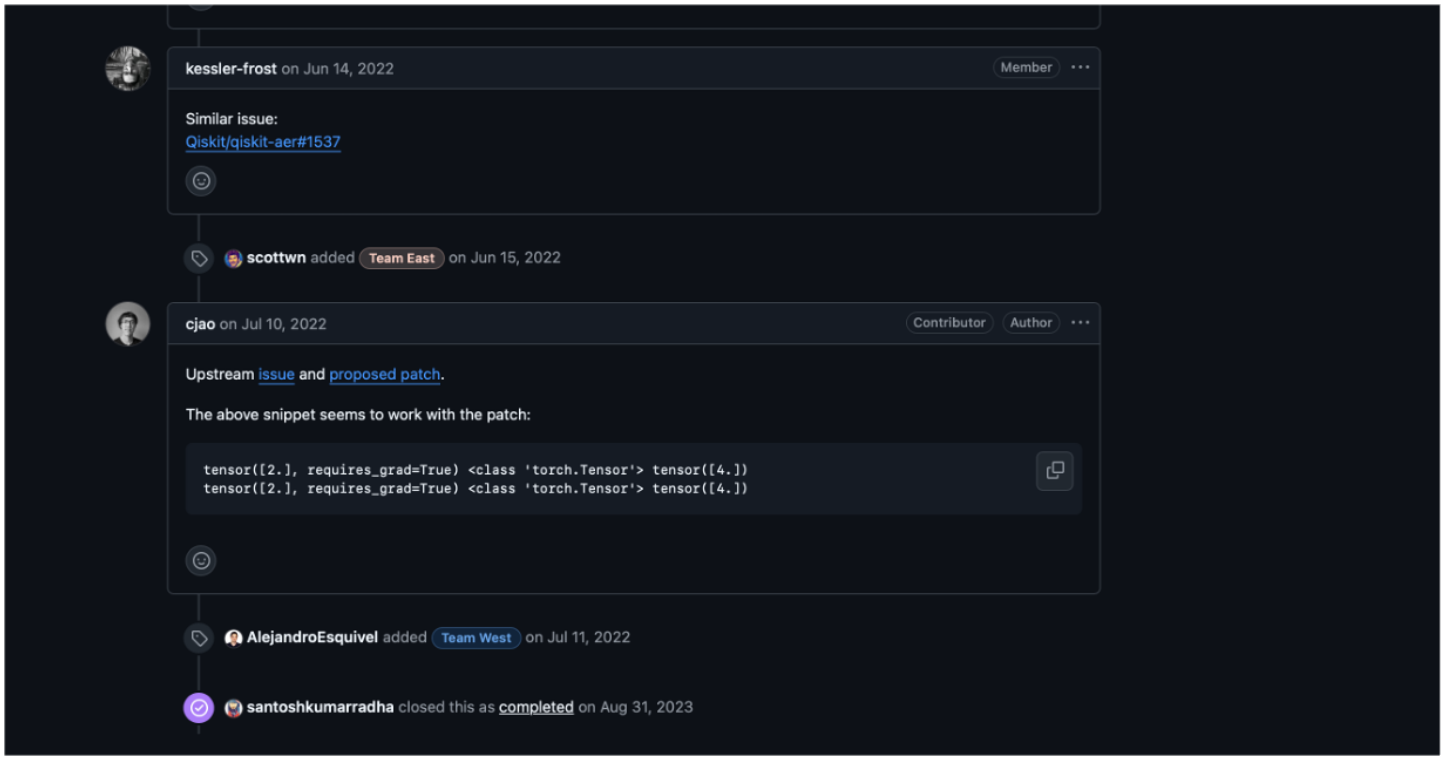
Milestone
No milestone

Relationships
None yet

Development
No branches or pull requests

Notifications — Customize
Subscribe
You're not receiving notifications from this thread.

Participants


kessler-frost on Jun 14, 2022
Member
Similar issue:
Qiskit/qiskit-aer#1537
scottwn added Team East on Jun 15, 2022
cjao on Jul 10, 2022
Contributor
Author
Upstream issue and proposed patch.
The above snippet seems to work with the patch:
tensor([2.], requires_grad=True) <class 'torch.Tensor'> tensor([4.])
tensor([2.], requires_grad=True) <class 'torch.Tensor'> tensor([4.])
AlejandroEsquivel added Team West on Jul 11, 2022
santoshkumarradha closed this as completed on Aug 31, 2023